

WAZUH-VT-THREATCHECK

**AN INTEGRATED FRAMEWORK FOR AUTOMATED THREAT
INTELLIGENCE INCIDENT RESPONSE**

SUBMITTED BY

SENHA FATHIMA

ABSTRACT

This project focuses on the deployment and configuration of **Wazuh**, an open-source Security Information and Event Management (SIEM) platform, integrated with multiple threat intelligence and monitoring tools **VirusTotal**, **AlienVault OTX**, **MITRE ATT&CK**, **File Integrity Monitoring (FIM)**, and **Active Response**.

The objective of this project is to establish a comprehensive and automated security monitoring environment capable of detecting, analyzing, and responding to threats in real time.

A **single-node Wazuh environment** was deployed using Docker Engine on Kali Linux 2026.2 and connected to a Kali Linux agent (hostname: BLACKICE) for active monitoring. **VirusTotal integration** enabled automatic analysis of file hashes to identify malware, while **AlienVault OTX** provided global threat intelligence to verify suspicious IPs via a custom Python integration script.

The **MITRE ATT&CK integration** mapped security events to attacker tactics and techniques, enhancing incident visibility. **FIM** ensured the integrity of system files by detecting unauthorized changes, and **Active Response** automated remediation actions upon detection of malicious activities.

Testing was conducted using simulated threats such as the **EICAR test file** and suspicious IP lookups to validate alerting and response mechanisms. The results confirmed that all integrations functioned as intended, providing enriched threat intelligence, improved detection accuracy, and automated mitigation.

TABLE OF CONTENT

NO.	CONTENTS	PAGE NO.
1	INTRODUCTION	4
2	OBJECTIVES	6
3	PREREQUISITES	7
4	RESEARCH METHODOLOGY	8-18
5	OBSERVATION AND RESULT	19
6	CONCLUSION	20

INTRODUCTION

1.1 Wazuh

Wazuh is an open-source security monitoring and threat detection platform designed for enterprise environments. It provides real-time log analysis, intrusion detection, file integrity monitoring, vulnerability detection, and compliance reporting. Wazuh helps organizations maintain security visibility across their infrastructure, allowing administrators to quickly identify and respond to potential threats. Its modular architecture and integration capabilities make it suitable for both small-scale and large-scale deployments.

1.2 File Integrity Monitoring (FIM)

File Integrity Monitoring (FIM) in Wazuh continuously checks for unauthorized changes in system files, configurations, and directories. It helps detect potential security breaches, configuration tampering, or malware infections by monitoring file creation, modification, and deletion events.

When a file change is detected, Wazuh generates alerts specifying the affected file path, type of change, and user responsible. FIM is crucial for compliance and forensic analysis, ensuring system integrity is maintained at all times. It plays a vital role in early detection of malicious activity, particularly in critical system directories such as /etc, /usr/bin, /usr/sbin, and /tmp.

1.3 VirusTotal Integration

VirusTotal is an online service that analyzes files, URLs, and IP addresses to detect viruses, malware, and other malicious content using multiple antivirus engines and threat intelligence tools. By providing detailed reports and file reputation data, VirusTotal enables users to identify potential security risks quickly. The service also offers an API, which allows automated integration with security platforms for real-time threat analysis.

Integrating Wazuh with VirusTotal enhances the security monitoring capabilities of Wazuh by automatically analyzing file hashes against VirusTotal's database. This integration allows Wazuh to generate alerts for potentially malicious files detected on monitored endpoints.

1.4 AlienVault OTX Integration

AlienVault Open Threat Exchange (OTX) is a collaborative threat intelligence platform that enables users to share and receive information about emerging cyber threats. OTX provides community-driven data known as "pulses," which contain indicators of compromise (IOCs) such as malicious IPs, domains, and file hashes.

Integrating Wazuh with AlienVault OTX allows automatic reputation checks of detected indicators against the OTX threat database. In this project, a custom Python script (custom-alienvault.py) was written and deployed inside the Wazuh manager container to perform IP reputation checks against the OTX API.

1.5 MITRE ATT&CK Integration

The MITRE ATT&CK framework is a globally recognized knowledge base that categorizes adversarial tactics, techniques, and procedures (TTPs) observed in real-world cyberattacks. It helps analysts understand the behavior of attackers and the progression of security incidents.

Integrating MITRE ATT&CK with Wazuh maps generated alerts to specific MITRE techniques and tactics. In this project, detections were automatically classified under MITRE IDs including T1565.001 (Data Manipulation), T1203 (Exploitation for Client Execution), T1078 (Valid Accounts), and T1548.003 (Sudo and Sudo Caching).

1.6 Active Response

Active Response in Wazuh provides automated remediation actions in response to specific alerts. Once a suspicious event or malicious activity is detected, Active Response scripts can execute predefined countermeasures such as blocking IP addresses, terminating processes, or isolating compromised endpoints.

This automation reduces the response time and limits the impact of attacks by containing threats immediately. In this project, the firewall-drop Active Response was configured to trigger automatically upon detection of malicious files identified by VirusTotal. During testing, the EICAR test file was successfully detected as malicious by 65 out of 67 antivirus engines generating Rule ID 87105, which triggered the firewall-drop response automatically, making it a proactive defense mechanism within the SOC environment.

OBJECTIVES

The main objectives of this project are to:

1. Set up a Wazuh SIEM environment using Docker Engine on Kali Linux to monitor and analyze security events.
2. Install and configure a Wazuh Agent on Kali Linux (BLACKICE) for endpoint monitoring.
3. Establish secure communication between the Wazuh Agent and Wazuh Manager to ensure accurate event collection and reporting.
4. Integrate VirusTotal for enhanced file and hash-based malware detection.
5. Integrate AlienVault OTX using a custom Python script for community-driven threat intelligence and real-time indicator reputation checks.
6. Enable MITRE ATT&CK mapping to classify detected threats based on adversarial tactics and techniques.
7. Implement File Integrity Monitoring (FIM) and Active Response to detect unauthorized system changes and automatically respond to security incidents.
8. Verify communication, detection accuracy, and automated alerting functionalities across all configured integrations.

PREREQUISITES

System Requirements

Single-node stack deployment:

- Operating system: Linux or Windows
- Architecture: AMD64
- CPU: At least 4 cores
- Memory: At least 8 GB of RAM
- Disk space: At least 50 GB storage

Wazuh agent deployment:

- Operating system: Linux or Windows
- Architecture: AMD64
- CPU: At least 2 cores
- Memory: At least 1 GB of RAM
- Disk space: At least 10 GB storage

System Used in This Project

Component	Details
Operating System	Kali Linux 2026.2 Rolling
Hostname	BLACKICE
Architecture	AMD64 (x86_64)
CPU	12 Cores
RAM	7.5 GB
Free Storage	101 GB
Docker Version	29.6.0
Wazuh Version	4.14.0

Required Software

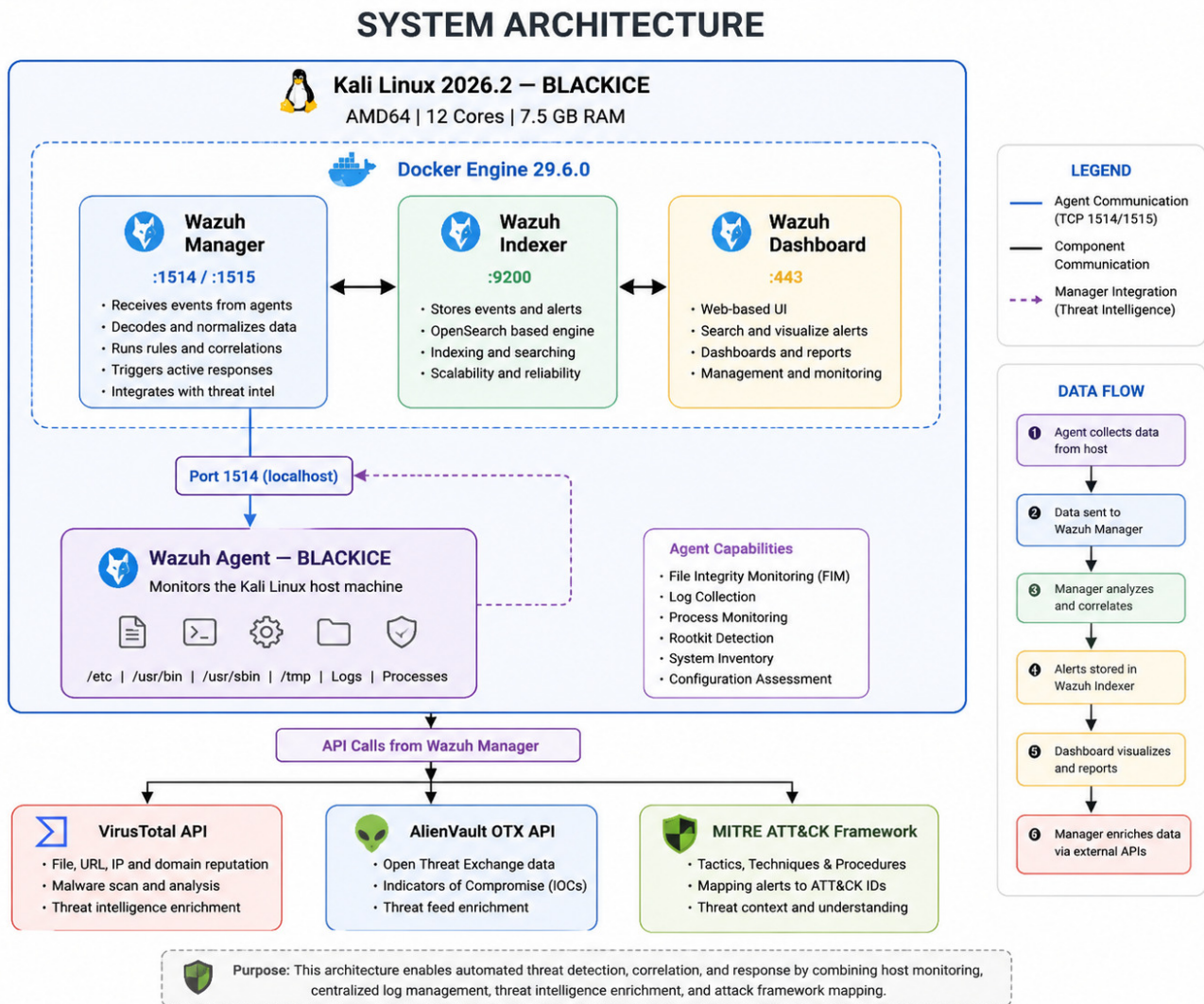
- Docker Engine 29.6.0
- Git
- Wazuh Agent 4.14.0
- Python 3 with requests module
- VirusTotal API Key
- AlienVault OTX API Key

RESEARCH METHODOLOGY

4.1 System Architecture

The proposed **WAZUH-VT-THREATCHHECK** framework provides centralized security monitoring by integrating the Wazuh platform with external threat intelligence services. **Figure 4.1** illustrates the overall system architecture, showing the interaction between the monitored Kali Linux host, the Docker-based Wazuh services, and external threat intelligence sources.

Figure 4.1: System Architecture of the WAZUH-VT-THREATCHHECK Framework



As shown in **Figure 4.1**, the framework is deployed on a **Kali Linux 2026.2** host machine. The **Wazuh Agent** is installed directly on the host operating system to monitor system files, log files, and running processes. It collects security events and forwards them to the **Wazuh Manager**, where event analysis, rule matching, and alert generation are performed.

To enhance detection accuracy, the **Wazuh Manager** integrates with the **VirusTotal API** and **AlienVault OTX API** to enrich events with external threat intelligence. In addition, detected activities are mapped to the **MITRE ATT&CK Framework**, providing contextual information about adversary tactics and techniques. The resulting alerts are stored in the **Wazuh Indexer**, while the **Wazuh Dashboard** provides a centralized interface for visualization, monitoring, and incident investigation.

This architecture clearly separates the host-based monitoring component from the Docker-deployed Wazuh services. This design enables comprehensive visibility of the Kali Linux host while maintaining a modular and maintainable system architecture. The integration of event processing, threat intelligence, and visualization improves overall threat detection and security monitoring capabilities.

4.2 Docker Engine Installation on Kali Linux

Docker Engine was installed on Kali Linux by adding the official Docker APT repository and installing the docker-ce package along with its required plugins. The Docker service was then enabled and started.

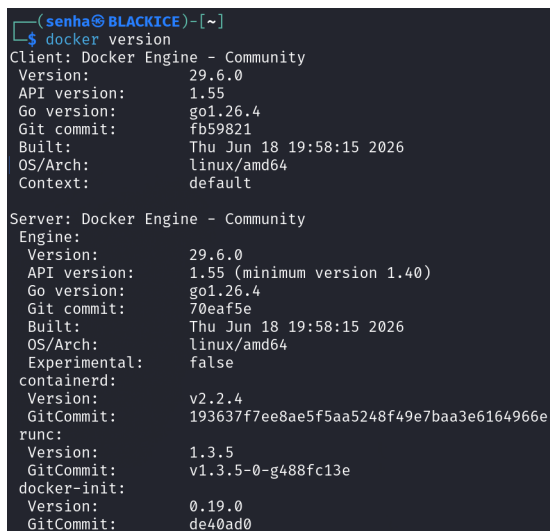
```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin -y
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Verification:

```
docker version
```



```
(senha@BLACKICE)-[~]
$ docker version
Client: Docker Engine - Community
Version: 29.6.0
API version: 1.55
Go version: go1.26.4
Git commit: fb59821
Built: Thu Jun 18 19:58:15 2026
OS/Arch: linux/amd64
Context: default

Server: Docker Engine - Community
Engine:
Version: 29.6.0
API version: 1.55 (minimum version 1.40)
Go version: go1.26.4
Git commit: 70eaf5e
Built: Thu Jun 18 19:58:15 2026
OS/Arch: linux/amd64
Experimental: false
containerd:
Version: v2.2.4
GitCommit: 193637f7ee8ae5f5aa5248f49e7baa3e6164966e
runc:
Version: 1.3.5
GitCommit: v1.3.5-0-g488fc13e
docker-init:
Version: 0.19.0
GitCommit: de40ad0
```

4.3 Git Installation and Wazuh Repository Clone

Git was installed and used to clone the official Wazuh Docker repository at version 4.14.0.

```
sudo apt install git -y
```

```
git clone https://github.com/wazuh/wazuh-docker.git -b v4.14.0
```

```
cd wazuh-docker/single-node/
```

```

(senha@BLACKICE)-[~]
$ git clone https://github.com/wazuh/wazuh-docker.git -b v4.14.0
Cloning into 'wazuh-docker'...
remote: Enumerating objects: 18053, done.
remote: Counting objects: 100% (1143/1143), done.
remote: Compressing objects: 100% (256/256), done.
remote: Total 18053 (delta 1051), reused 892 (delta 887), pack-reused 16910 (from 3)
Receiving objects: 100% (18053/18053), 7.58 MiB | 2.90 MiB/s, done.
Resolving deltas: 100% (9930/9930), done.
warning: refs/tags/v4.14.0 ac6381693e20ee229a613c8470c7c1f2ada6b3b0 is not a commit!
Note: switching to '4c7ee8abac3d359084d63f1452387c105d00d0a5'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

```

```

(senha@BLACKICE)-[~]
$ cd wazuh-docker/single-node/

(senha@BLACKICE)-[~/wazuh-docker/single-node]
$ ls
config  docker-compose.yml  generate-indexer-certs.yml  README.md

```

4.4 SSL Certificate Generation

SSL certificates required for secure communication between Wazuh components were generated using the certificate generator provided in the Wazuh Docker repository.

`docker compose -f generate-indexer-certs.yml run --rm generator`

```

17d0386c2fff Extracting 1B
17d0386c2fff Extracting 1B
17d0386c2fff Extracting 1B
17d0386c2fff Extracting 1B
17d0386c2fff Extracting 1B
7ce91ec7d1d3 Extracting 1B
17d0386c2fff Pull complete 0B
7ce91ec7d1d3 Extracting 1B
7ce91ec7d1d3 Extracting 1B
7ce91ec7d1d3 Extracting 1B
7ce91ec7d1d3 Extracting 1B
7ce91ec7d1d3 Extracting 1B
7ce91ec7d1d3 Pull complete 0B
d7003467fd14 Pull complete 0B
5249716d429c Pull complete 0B
Image wazuh/wazuh-certs-generator:0.0.2 Pulled
Container single-node-generator-run-76a982e1d95f Creating
Container single-node-generator-run-76a982e1d95f Created
The tool to create the certificates exists in the in Packages bucket
19/06/2026 05:24:37 INFO: Generating the root certificate.
19/06/2026 05:24:37 INFO: Generating Admin certificates.
19/06/2026 05:24:37 INFO: Admin certificates created.
19/06/2026 05:24:37 INFO: Generating Wazuh indexer certificates.
19/06/2026 05:24:37 INFO: Wazuh indexer certificates created.
19/06/2026 05:24:37 INFO: Generating Filebeat certificates.
19/06/2026 05:24:37 INFO: Wazuh Filebeat certificates created.
19/06/2026 05:24:37 INFO: Generating Wazuh dashboard certificates.
19/06/2026 05:24:37 INFO: Wazuh dashboard certificates created.
Moving created certificates to the destination directory
Changing certificate permissions
Setting UID indexer and dashboard
Setting UID for wazuh manager and worker

```

4.5 Wazuh Deployment Using Docker

The Wazuh single-node stack was deployed using Docker Compose. All three containers wazuh.manager, wazuh.indexer, and wazuh.dashboard were started and verified.

```
docker compose up -d
```

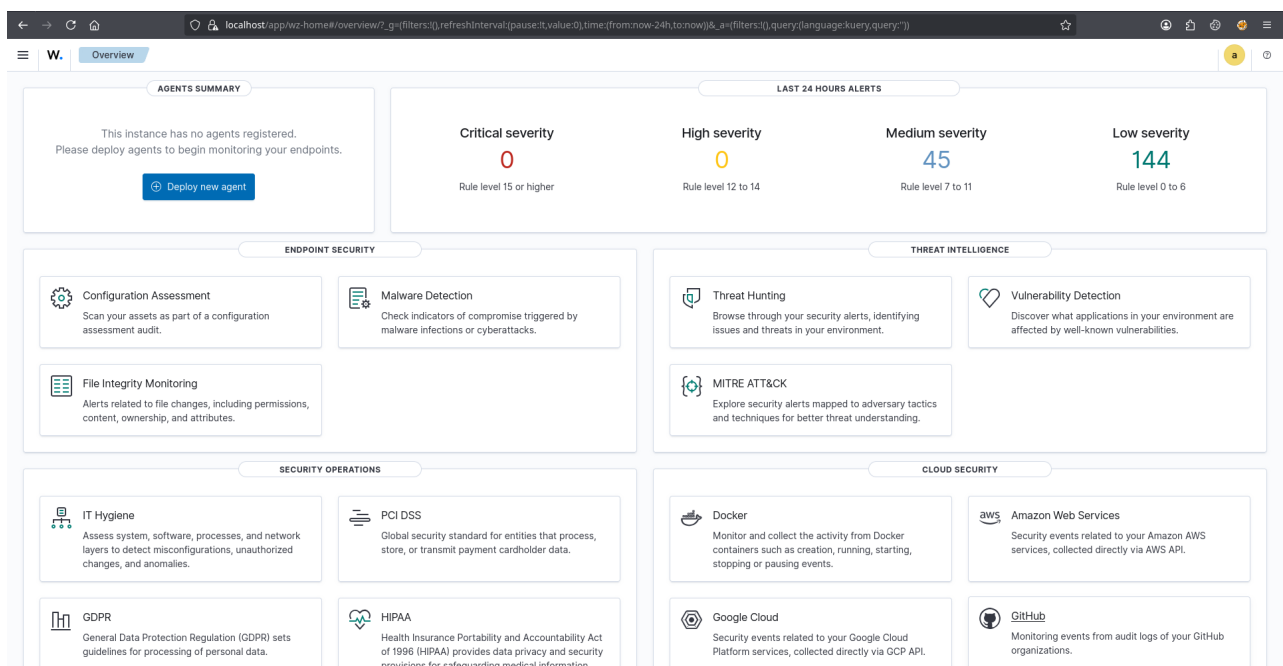
```
docker ps
```

```
(senha@BLACKICE) - [~/wazuh-docker/single-node]
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAMES
ac7e0545d222   wazuh/wazuh-dashboard:4.14.0       "/entrypoint.sh"        4 minutes ago Up About a minute  443/tcp, 0.0.0.0:443->5601/
tcp, [::]:443->5601/tcp
single-node-wazuh.dashboard-1
d8a7b9f032c3   wazuh/wazuh-indexer:4.14.0         "/entrypoint.sh open..." 4 minutes ago  Up About a minute
single-node-wazuh.indexer-1
ffd604ca319a   wazuh/wazuh-manager:4.14.0         "/init"                  4 minutes ago Up 4 minutes      0.0.0.0:1514-1515->1514-151
5/tcp, [::]:1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]:55000->55000/tcp, 1516/t
cp single-node-wazuh.manager-1
```

4.6 Accessing the Wazuh Dashboard

After deployment, the Wazuh dashboard was accessed via browser at <https://localhost> using the default credentials.

- Username: admin
- Password: SecretPassword



4.7 Wazuh Agent Installation on Kali Linux

The Wazuh agent was installed on the same Kali Linux machine (BLACKICE) by adding the official Wazuh APT repository. The manager address was configured to 127.0.0.1 in the agent configuration file.

```
WAZUH_MANAGER="127.0.0.1" sudo apt install wazuh-agent -y
```

```
sudo systemctl enable wazuh-agent
```

```
sudo systemctl start wazuh-agent
```

Verification:

```
sudo /var/ossec/bin/wazuh-control status
```

```
(senha@BLACKICE)-[~/wazuh-docker/single-node]
$ sudo systemctl restart wazuh-agent

(senha@BLACKICE)-[~/wazuh-docker/single-node]
$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-06-19 11:45:29 IST; 3s ago
     Invocation: 8655591d896c4e5fac540e6f32a04b67
   Process: 67548 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 42 (limit: 8947)
   Memory: 891.7M (peak: 911.4M)
     CPU: 15.152s
   CGroup: /system.slice/wazuh-agent.service
           └─67570 /var/ossec/bin/wazuh-execd
             └─67590 /var/ossec/bin/wazuh-agentd
               └─67605 /var/ossec/bin/wazuh-syscheckd
                 └─67618 /var/ossec/bin/wazuh-logcollector
                   └─67632 /var/ossec/bin/wazuh-modulesd
                     └─67878 iptables -L

Jun 19 11:45:23 BLACKICE systemd[1]: Starting wazuh-agent.service - Wazuh agent...
Jun 19 11:45:23 BLACKICE env[67548]: Starting Wazuh v4.14.5...
Jun 19 11:45:24 BLACKICE env[67548]: Started wazuh-execd...
Jun 19 11:45:25 BLACKICE env[67548]: Started wazuh-agentd...
Jun 19 11:45:26 BLACKICE env[67548]: Started wazuh-syscheckd...
Jun 19 11:45:26 BLACKICE env[67548]: Started wazuh-logcollector...
Jun 19 11:45:27 BLACKICE env[67548]: Started wazuh-modulesd...
Jun 19 11:45:29 BLACKICE env[67548]: Completed.
Jun 19 11:45:29 BLACKICE systemd[1]: Started wazuh-agent.service - Wazuh agent.
```

4.8 File Integrity Monitoring (FIM)

Purpose:

FIM detects unauthorized changes in files and directories.

Steps:

The agent configuration file `/var/ossec/etc/ossec.conf` was edited to add the directories to be monitored under the `<syscheck>` block. The scan frequency was set to 300 seconds for faster detection during testing.

```
<syscheck>
```

```
<disabled>no</disabled>
```

```
<frequency>300</frequency>
```

```
<scan_on_start>yes</scan_on_start>
```

```
<directories>/etc,/usr/bin,/usr/sbin</directories>
```

```
<directories>/bin,/sbin,/boot</directories>
```

```
<directories>/tmp</directories>
```

```
</syscheck>
```

The Wazuh agent was restarted to apply the configuration:

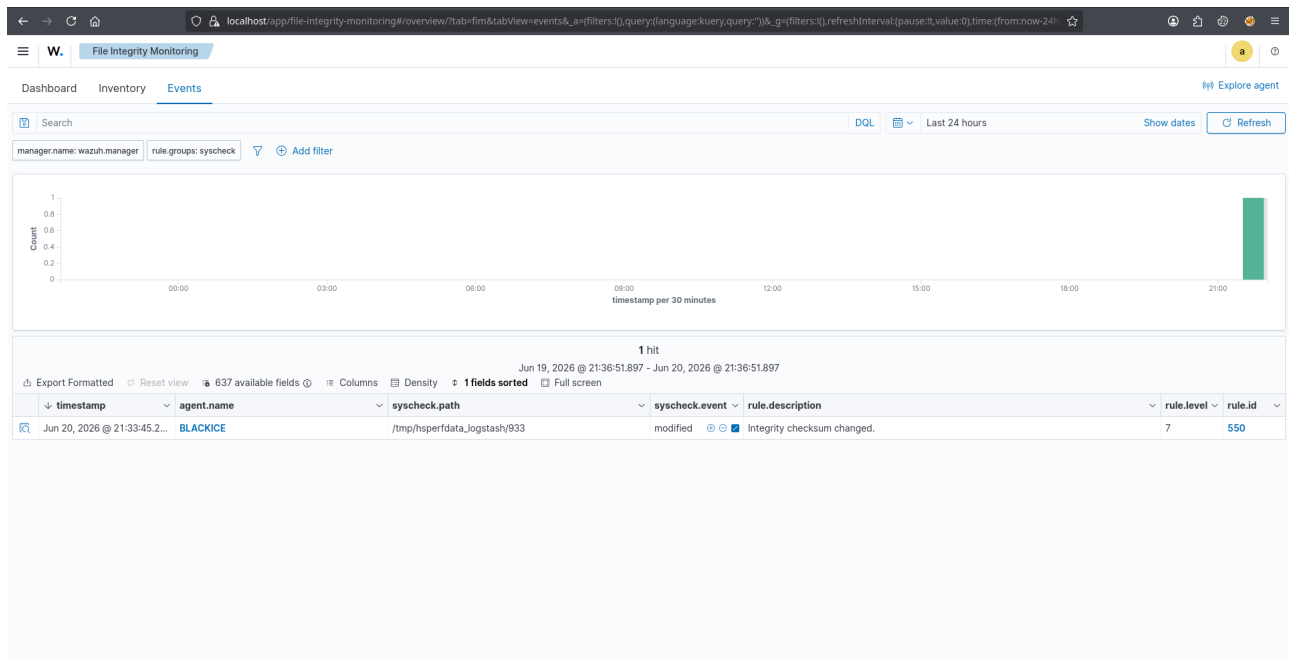
```
sudo systemctl restart wazuh-agent
```

A test file was created and deleted to trigger a FIM detection:

```
sudo touch /tmp/testfile.txt
```

```
sudo rm /tmp/testfile.txt
```

Alerts were verified under File Integrity Monitoring → Events in the dashboard.



4.9 VirusTotal Integration

Purpose:

VirusTotal integration automatically analyzes file hashes detected by FIM against VirusTotal's global malware database.

Steps:

A VirusTotal account was created at <https://www.virustotal.com> and the API key was obtained from Profile → API Key.

The following integration block was added to `/var/ossec/etc/ossec.conf` inside the Wazuh manager container:

```
<ossec_config>

<integration>

<name>virustotal</name>

<api_key>YOUR_API_KEY</api_key>

<group>syscheck</group>

<alert_format>json</alert_format>

</integration>

</ossec_config>
```

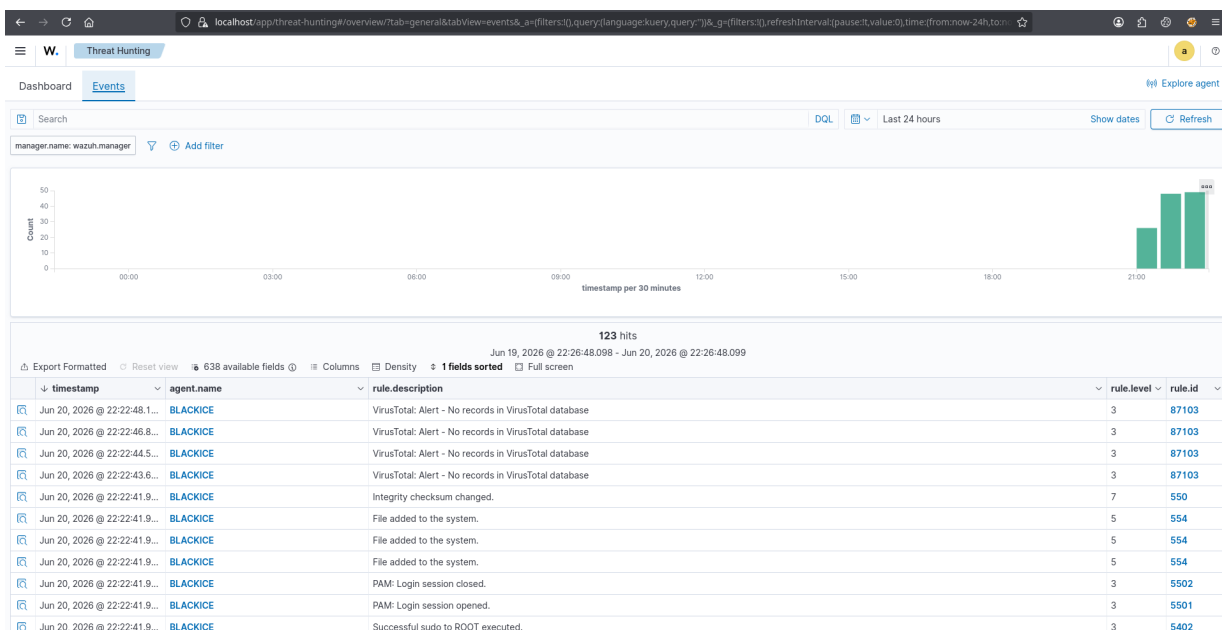
The Wazuh manager was restarted to apply the configuration:

```
docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/wazuh-control restart
```

The EICAR test file was downloaded to trigger a VirusTotal detection:

```
curl -o /tmp/eicar.com.txt https://secure.eicar.org/eicar.com.txt
```

Alerts were verified under Threat Hunting → Events in the dashboard.



The VirusTotal integration was verified under Threat Hunting → Events. Wazuh successfully queried the VirusTotal API for every file hash detected by FIM, generating 123 alerts. Most alerts showed Rule ID 87103 due to the free API rate limit of 4 requests per minute. However, the EICAR test file was successfully detected as malicious by 65 out of 67 antivirus engines generating Rule ID 87105, which also triggered the Active Response firewall-drop script automatically.

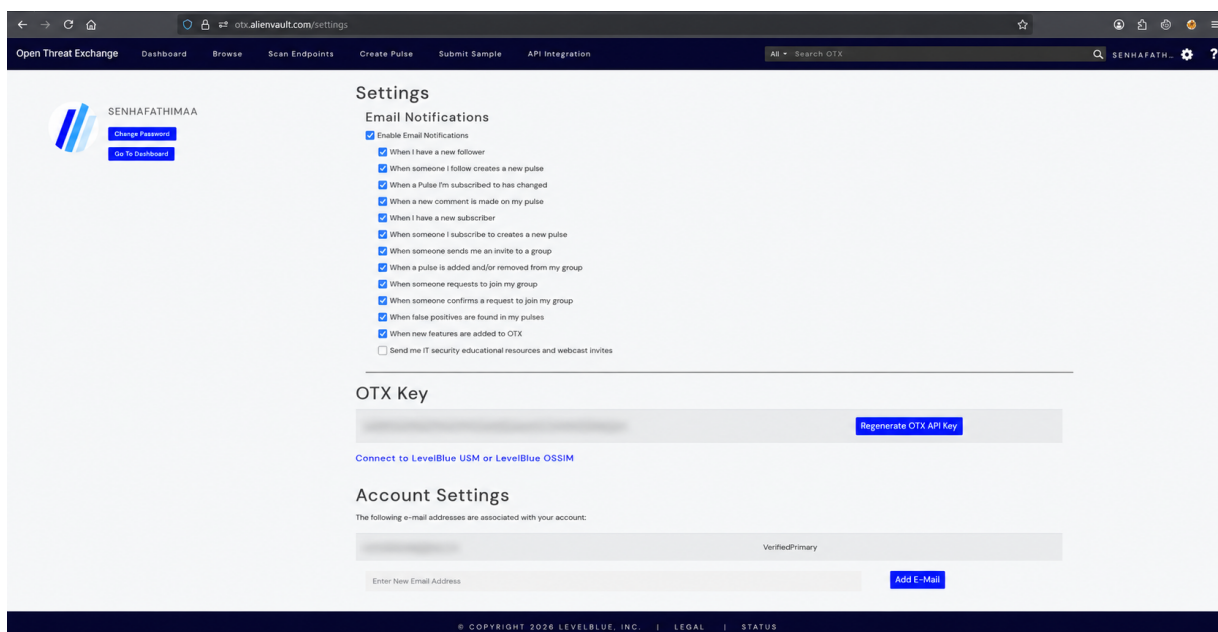
4.10 AlienVault OTX Integration

Purpose:

AlienVault OTX provides real-time IP reputation checks against global threat intelligence data.

Steps:

An OTX account was created at <https://otx.alienvault.com> and the API key was obtained from Profile → API Key.



A custom Python integration script (**custom-alienvault.py**) was created and deployed inside the Wazuh manager container to perform IP reputation checks against the OTX API. The **requests** module was installed inside the container to support the script.

The following integration block was added to **ossec.conf**:

```
<ossec_config>

<integration>

<name>custom-alienvault</name>

<hook_url></hook_url>

<level>3</level>
```

```
<alert_format>json</alert_format>
```

```
</integration>
```

```
</ossec_config>
```

The integration was tested by submitting a suspicious IP to the script:

```
docker exec -it single-node-wazuh.manager-1 /bin/bash -c \
```

```
'echo "{\"data\":{\"srcip\":\"198.51.100.10\"}}\" | python3 /var/ossec/integrations/custom-alienvault.py'
```

```
(senha@BLACKICE)-[~/wazuh-docker/single-node]
$ docker exec -it single-node-wazuh.manager-1 /bin/bash -c 'echo "{\"data\":{\"srcip\":\"198.51.100.10\"}},\"rule\":{\"id\":\"100010\",\"level\":\"10\",\"full_log\":\"test\"}\" | python3 /var/ossec/integrations/custom-alienvault.py'
AlienVault OTX Reputation Check for 198.51.100.10:
Pulses: 0
Reputation: 0
Country: unknown
OTX Clean: 198.51.100.10 not found in any pulses.
```

4.11 MITRE ATT&CK Integration

Purpose:

MITRE ATT&CK maps alerts to real-world attacker tactics and techniques for better incident context.

Steps:

MITRE ATT&CK is built into Wazuh by default. Alerts were generated by performing the following activities on the Kali Linux agent:

```
whoami
```

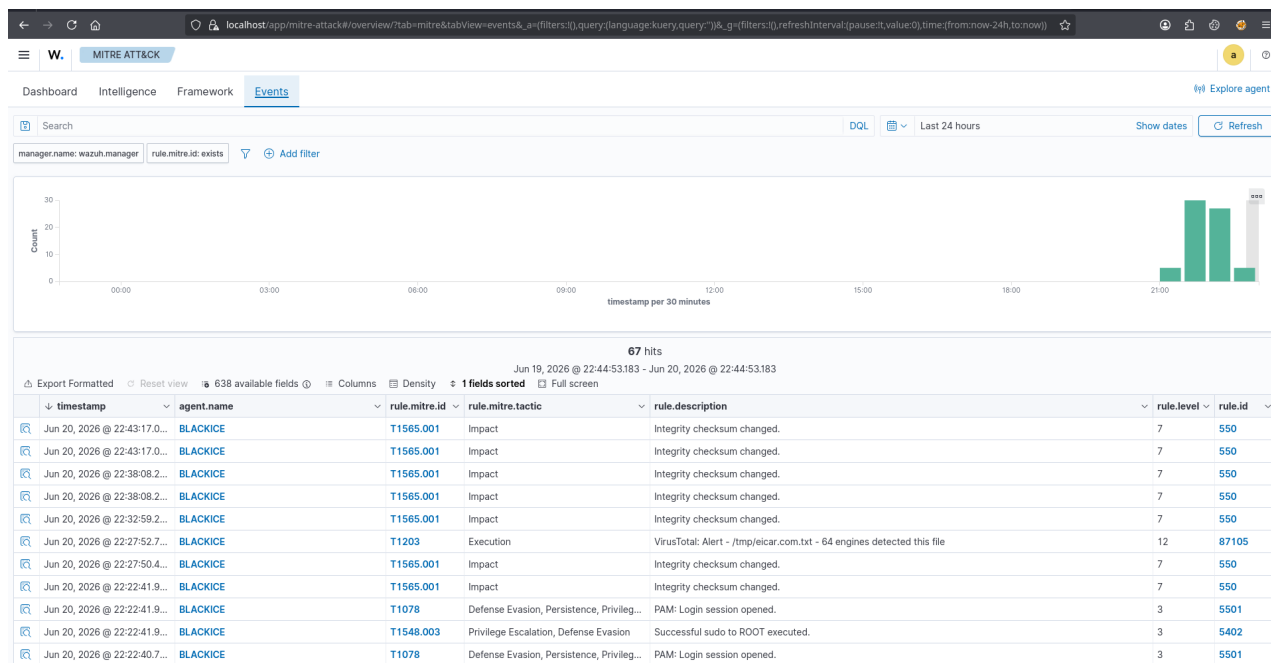
```
id
```

```
cat /etc/passwd
```

```
netstat -an
```

```
ps aux
```

Results were viewed under MITRE ATT&CK → Overview filtered by agent BLACKICE.



4.12 Active Response

Purpose:

Active Response automatically executes the firewall-drop script upon detection of malicious files by VirusTotal.

Steps:

The following Active Response block was added to `/var/ossec/etc/ossec.conf`:

```
<ossec_config>

<active-response>

<command>firewall-drop</command>

<location>local</location>

<rules_id>87105</rules_id>

<timeout>600</timeout>

</active-response>

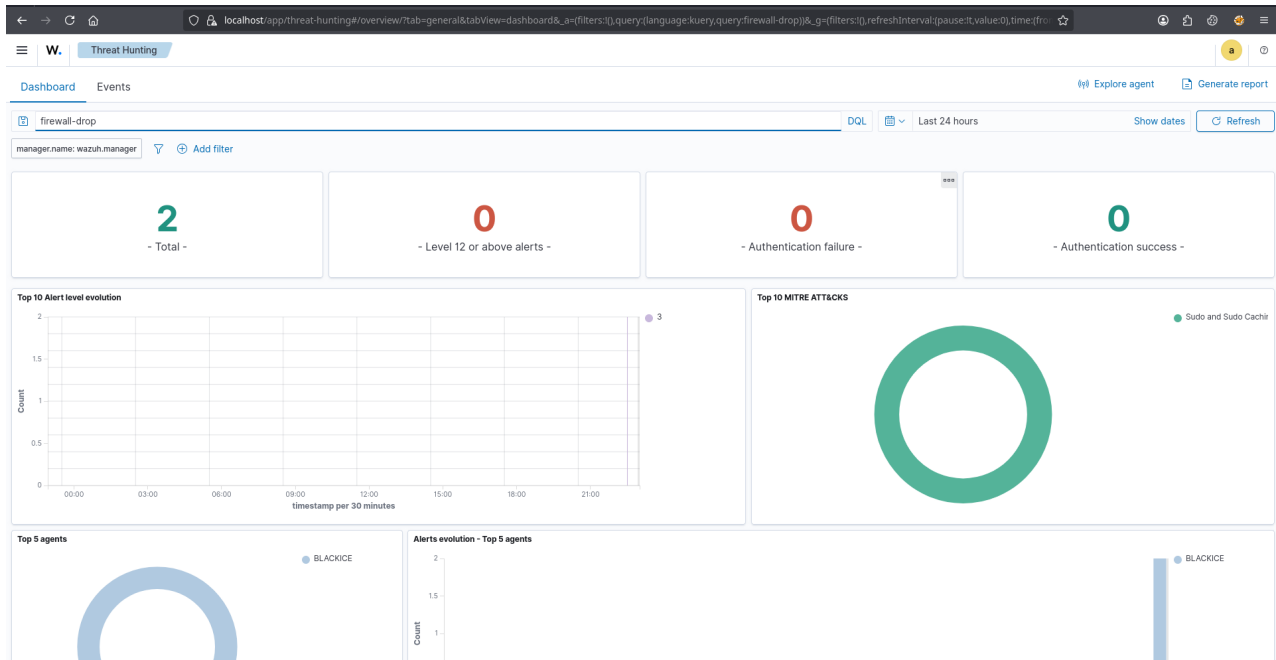
</ossec_config>
```

The Wazuh manager was restarted and the EICAR file was downloaded again to trigger the response. The Active Response logs were checked to confirm execution:

```
sudo grep "firewall-drop" /var/ossec/logs/active-responses.log
```

Results were verified under Threat Hunting → Dashboard by searching for "firewall-drop".

```
(senha@BLACKICE)-[~/wazuh-docker/single-node]
$ sudo grep "firewall-drop" /var/ossec/logs/active-responses.log
[sudo] password for senha:
2026/06/20 22:53:42 active-response/bin/firewall-drop: Starting
2026/06/20 22:53:42 active-response/bin/firewall-drop: {"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"extra_ar
gs":[],"alert":{"timestamp":"2026-06-20T17:23:42.880+0000"},"rule":{"level":12,"description":"VirusTotal: Alert - /tmp/eicar-test.com.txt - 65 engines detecte
d this file","id":"87105","mitre":{"id":["T1203"],"tactic":["Execution"],"technique":["Exploitation for Client Execution"]},"firedtimes":1,"mail":true,"group
s":["virustotal"],"pci_dss":["10.6.1","11.4"],"gdpr":["IV_35.7.d"]},"agent":{"id":"001","name":"BLACKICE","ip":"127.0.0.1"},"manager":{"name":"wazuh.manager"
},"id":"1781976222.112018","decoder":{"name":"json"},"data":{"virustotal":{"found":"1","malicious":"1","source":{"alert_id":"1781976219.110181","file":"/tmp/
eicar-test.com.txt","md5":"44d88612fea8a8f36de82e1278abb02f","sha1":"3395856ce81f2b7382dee72602f798b642f14140"},"sha1":"3395856ce81f2b7382dee72602f798b642f14
140","scan_date":"2026-06-20 17:12:06","positives":"65","total":"67","permalink":"https://www.virustotal.com/gui/file/275a021bbfb6489e54d471899f7db9d1663fc69
5ec2fe2a2c4538aabf651fd0f/detection/f-275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c4538aabf651fd0f-1781975526"},"integration":"virustotal"},"location":"v
irustotal"},"program":"active-response/bin/firewall-drop"}}
2026/06/20 22:53:42 active-response/bin/firewall-drop: Cannot read 'srcip' from data
```



OBSERVATION AND RESULT

After completing the configuration and integration process, the Wazuh environment successfully monitored the connected Kali Linux agent (BLACKICE). The Wazuh Dashboard displayed real-time event data, enabling efficient visualization of system activity and threat alerts.

The VirusTotal integration functioned effectively by automatically analyzing file hashes detected by FIM. Wazuh successfully queried the VirusTotal API for every file change detected, generating 123 alerts. Most alerts returned Rule ID 87103 due to the free API rate limit. However, the EICAR test file was successfully identified as malicious by 65 out of 67 antivirus engines, generating Rule ID 87105 and confirming that the VirusTotal API integration was fully operational.

The AlienVault OTX integration provided real-time threat intelligence via a custom Python script deployed inside the Wazuh manager container. When a test IP (198.51.100.10) was submitted, Wazuh successfully queried the OTX API and returned pulse count, reputation, and country data, validating successful communication with the AlienVault API.

The MITRE ATT&CK integration accurately mapped alerts to their respective tactics and techniques: T1565.001 (Impact), T1203 (Execution), T1078 (Defense Evasion, Persistence), and T1548.003 (Privilege Escalation, Defense Evasion). This mapping enhanced the contextual understanding of security incidents and allowed visualization of attacker behavior patterns in the MITRE ATT&CK tab.

The File Integrity Monitoring feature effectively detected file changes in monitored directories including /etc, /usr/bin, /usr/sbin, and /tmp. Alerts were generated with Rule ID 550 (Integrity checksum changed) and Rule ID 554 (File added to the system), confirming FIM's role in maintaining system integrity.

The Active Response module demonstrated Wazuh's ability to perform automated defensive actions. Upon detection of the EICAR test file as malicious by VirusTotal, Wazuh triggered the predefined firewall-drop Active Response script automatically. This was confirmed in the active-responses.log file, validating that the automated mitigation mechanism functioned as expected.

Overall, all configured integrations operated as expected, validating the effectiveness of the Wazuh-VT-ThreatCheck framework as a complete threat intelligence and incident response solution. Wazuh's centralized monitoring, combined with these external intelligence feeds and automated responses, significantly enhanced the environment's detection accuracy and response capability.

CONCLUSION

The successful deployment and integration of Wazuh with VirusTotal, AlienVault OTX, MITRE ATT&CK, File Integrity Monitoring (FIM), and Active Response on Kali Linux 2026.2 using Docker Engine demonstrates a comprehensive and practical approach to real-time threat detection, analysis, and remediation.

Through VirusTotal, Wazuh effectively enriched file-based alerts with global malware intelligence by automatically analyzing file hashes detected by FIM against the VirusTotal database. The EICAR test file was successfully identified as malicious by 65 out of 67 antivirus engines, generating Rule ID 87105 and triggering the Active Response firewall-drop script automatically.

Unlike traditional Windows-based deployments, this project was implemented entirely on Kali Linux 2026.2 using Docker Engine, providing native performance and a realistic SOC environment aligned with industry practices. The AlienVault OTX integration was also implemented as a custom Python script rather than a built-in connector, demonstrating the extensibility and flexibility of the Wazuh platform.

This combination transformed Wazuh from a standard SIEM into a proactive, intelligence-driven security solution capable of identifying, correlating, and responding to threats efficiently. The project validates Wazuh's capability to serve as an essential component in a Security Operations Center (SOC) setup, providing scalability, automation, and deep visibility across endpoints.

In the future, this environment can be further enhanced by integrating machine learning-based anomaly detection, custom dashboards, and automated playbooks, thereby strengthening proactive defense, incident correlation, and overall cybersecurity resilience.